Data Access Patterns

> **Document Owner:** Platform Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

The Databricks Lakehouse platform provides three distinct capabilities for making external data available to analytics workloads. Customers frequently ask: *"Should I ingest, federate, or share?"* This document provides a decision framework with golden rules to ensure the right pattern is applied for each external data source.

Pattern	Mechanism	When to Use
Ingestion (Lakeflow Connect)	Copy data into Delta Lake	Production analytics, dashboards, ML
Lakehouse Federation	Query external DBs in place via JDBC or catalog access	Ad-hoc exploration, migration staging, POC
Delta Sharing	Zero-copy sharing across orgs/metastores	Cross-organization exchange, data mesh

Key principle: Ingestion is the default for production. Federation and sharing are complements, not replacements.

Golden Rules

ID	Rule	Severity	WAF Pillar
DI-01	Default to ingestion for production analytics	Critical	Performance
DI-02	Reserve federation for ad-hoc access and migration staging	Critical	Performance
DI-03	Use Delta Sharing for cross-organization data exchange	Required	Interop
DI-04	Prefer catalog federation over query federation	Required	Performance
DI-05	Transition federation to ingestion when patterns stabilize	Required	OpEx
DI-06	Document the data access pattern for every external source	Required	Governance
DI-07	Never federate high-volume joins or aggregations	Critical	Performance

> **Cross-references:** Ingestion tools — see DP-05, DP-06. Federation setup — see PA-04. Sharing protocols — see DS-01..07.

Data Access Pattern Comparison

Factor	Ingestion (Lakeflow Connect)	Federation (Lakehouse)	Delta Sharing
Data location	Copied to Delta Lake	Stays in source system	Stays with provider
Query performance	Full Delta optimizations (Photon, clustering, caching)	JDBC pushdown or catalog storage access	Near real-time reads from provider's Delta tables
Data freshness	Depends on schedule (near-RT to daily batch)	Always live (queries source directly)	Always live (reads provider's current data)
Volume suitability	High volume, frequent queries	Low volume, ad-hoc (<1M rows/query)	Any volume (zero-copy)
Governance	Full UC: lineage, tags, PK/FK, quality expectations, monitors	UC: lineage, permissions, tags over foreign catalog	UC shares: provider controls access, audit on both sides
Transformations	Full Medallion pipeline (Bronze/Silver/Gold)	Read-only queries (no writes back to source)	Read-only on recipient side (can CTAS into local tables)
Cost model	Storage + compute (your cluster)	Compute only (your cluster + load on source system)	Minimal: no replication, no egress within region
Supported sources	Salesforce, ServiceNow, SQL Server, Workday, files, Kafka, Google Analytics, SharePoint	PostgreSQL, MySQL, SQL Server, Snowflake, BigQuery, Redshift, Oracle, Teradata, Synapse	Any UC metastore (D2D), any platform (open sharing)
Primary use cases	Production analytics, ML features, dashboards, SLAs	Migration staging, ad-hoc exploration, POC, operational lookups	Cross-org collaboration, marketplace, data mesh, multi-cloud
Who controls data	You (full copy, full governance)	Source system owner (you have read-only view)	Provider (share owner controls what's visible)
Delta Lake features	All: time travel, CDF, clustering, quality	None (foreign tables)	Most: time travel, CDF, streaming (D2D)
Write-back capability	Full (MERGE, INSERT, UPDATE, DELETE)	No	No (read-only for recipients)

Decision Tree

```
External data source identified
│
├── Data owned by another organization or team on a separate UC metastore?
│   └── YES → Delta Sharing (DI-03)
│       ├── Both parties on Databricks UC? → Databricks-to-Databricks
sharing (DS-02)
│       └── Recipient not on Databricks? → Open sharing with tokens (DS-
06)
│
├── Data in an external RDBMS or data warehouse (you have credentials)?
│   ├── Need for production dashboards, ML, or recurring pipelines?
│   │   └── YES → Ingest into Delta Lake (DI-01)
│   │       └── Has Lakeflow Connect managed connector? → Yes: DP-05 / No:
DLT + Auto Loader
│   ├── Exploratory, ad-hoc, or POC with low query volume?
│   │   └── YES → Federate (DI-02)
│   │       ├── Source supports catalog federation? → Catalog federation
(DI-04)
│   │       └── Otherwise → Query federation via JDBC (PA-04)
│   └── Migrating from legacy system?
│       └── Federate first for continuity (DI-02), then ingest as patterns
stabilize (DI-05)
│
├── Data in cloud storage (S3, ADLS, GCS)?
│   └── Ingest with Auto Loader or Lakeflow Connect file connectors (DP-
05)
│
└── Data from SaaS APIs (Salesforce, ServiceNow, Workday, etc.)?
    └── Ingest with Lakeflow Connect managed connector (DP-05)
```

DI-01: Default to Ingestion for Production Analytics

Rule

Data must reside in Delta Lake for production analytics workloads. Use Lakeflow Connect managed connectors first, then DLT + Auto Loader, then Structured Streaming. Only deviate when federation or sharing provides a documented advantage.

Why It Matters

- Delta Lake enables Photon acceleration, liquid clustering, predictive optimization, and time travel
- Full Unity Catalog governance: lineage, tags, PK/FK constraints, quality expectations
- Medallion Architecture (Bronze/Silver/Gold) only works with ingested data
- SLAs cannot be guaranteed on federated sources (source system latency is unpredictable)

Ingestion Priority

Priority	Approach	Best For	Key Feature
1	**Lakeflow Connect**	Salesforce, ServiceNow, SQL Server, Workday, SharePoint, Google Analytics	Fully managed: CDC, schema evolution, retries
2	**DLT + Auto Loader**	File-based sources (S3/ADLS/GCS), message buses (Kafka, Kinesis, Pub/Sub)	Declarative, incremental, expectations
3	**Structured Streaming**	Complex real-time with custom logic	Full programmatic control

Correct

```yaml
# Lakeflow Connect managed connector (preferred)
resources:
  pipelines:
    salesforce_ingest:
      name: "[${bundle.target}] Salesforce Ingest"
      serverless: true
      catalog: ${var.catalog}
      target: bronze
      ingestion_definition:
        connection_name: salesforce_prod
        objects:
          - table:
              source_schema: sobjects
              source_table: Account
              destination_catalog: ${var.catalog}
              destination_schema: bronze
```

Anti-Pattern

```sql
-- WRONG: Querying Salesforce via federation for a daily dashboard
-- This puts load on Salesforce API, has unpredictable latency,
-- and bypasses all Delta Lake optimizations
SELECT account_name, SUM(amount)
FROM salesforce_federated.sobjects.opportunity
WHERE close_date >= current_date() - 30
GROUP BY 1;
```

DI-02: Reserve Federation for Ad-Hoc Access

Rule

Use Lakehouse Federation only for: (1) ad-hoc exploration of external databases, (2) low-volume operational lookups (<1M rows per query, <10 queries/day), (3) migration staging where you need

immediate access while building ingestion pipelines. Never use federation as the primary data source for production dashboards or recurring pipelines.

Why It Matters

- Federation queries traverse JDBC connections with higher latency and lower throughput than local Delta reads
- Query pushdown is limited — complex joins, window functions, and UDFs may not push down, causing full table scans over the network
- Source system load — every federated query hits the source database, which may have its own concurrency limits
- No Delta features — no time travel, no CDF, no clustering, no quality expectations

When to Federate

Scenario	Federate?	Reasoning
Exploring a Snowflake dataset for migration	Yes	Short-term access while building ingestion
Production executive dashboard	**No**	Ingest — need SLA guarantees
One-time ad-hoc query on PostgreSQL	Yes	Low volume, no recurring need
Joining federated + local Delta tables	**No**	Performance cliff — ingest the external data
Operational lookup (e.g., user validation)	Maybe	Only if <1M rows and infrequent

Correct

```sql
-- Ad-hoc exploration of external PostgreSQL during migration assessment
CREATE CONNECTION legacy_crm TYPE POSTGRESQL
OPTIONS (
    host 'crm-db.company.com',
    port '5432',
    user secret('federation-scope', 'pg-user'),
    password secret('federation-scope', 'pg-password')
);

CREATE FOREIGN CATALOG legacy_crm USING CONNECTION legacy_crm
OPTIONS (database 'crm_production');

-- Quick exploration - fine for federation
SELECT COUNT(*), MIN(created_at), MAX(created_at)
FROM legacy_crm.public.customers;
```

Anti-Pattern

```sql
-- WRONG: Recurring pipeline reading from federated source
-- This runs daily, joins with local Delta, and aggregates - should be
ingested
```

```sql
CREATE OR REPLACE TABLE gold.customer_360 AS
SELECT c.*, o.total_orders, o.total_revenue
FROM legacy_crm.public.customers c
JOIN gold.fact_orders o ON c.customer_id = o.customer_id;
-- Fix: Ingest legacy_crm.public.customers into bronze first
```

DI-03: Delta Sharing for Cross-Organization Exchange

Rule

When data is owned by another organization, business unit on a separate Unity Catalog metastore, or an external partner, use Delta Sharing for zero-copy, governed access. Do not replicate data that can be shared — it creates governance gaps and staleness.

Why It Matters

- Zero data replication — no storage duplication, no egress fees within region
- Provider retains control — can revoke access instantly, filter rows/columns via dynamic views
- Full audit trail — both provider and recipient can monitor access via system tables
- Cross-cloud capable — share between AWS, Azure, and GCP

When to Share vs Ingest

Scenario	Pattern	Reasoning
Partner provides daily sales data	Delta Sharing	Partner controls data, you consume read-only
Another BU on separate metastore shares reference data	Delta Sharing (D2D)	Zero-copy, governed, no duplication
You pull data from your own Snowflake DW	Ingest	You own and control the source
External vendor sends CSV files	Ingest	No Delta Sharing support for raw files

Correct

```sql
-- Recipient: Access data shared by another business unit (D2D)
CREATE CATALOG IF NOT EXISTS partner_data
USING SHARE partner_org.sales_share;

-- Query shared data directly — always fresh, zero copy
SELECT region, SUM(revenue)
FROM partner_data.sales.fact_orders
WHERE order_date >= '2026-01-01'
GROUP BY 1;
```

Anti-Pattern

```
# WRONG: Copying shared data into your own tables unnecessarily
# This duplicates data, loses freshness, and creates governance gaps
shared_df = spark.table("partner_data.sales.fact_orders")
shared_df.write.mode("overwrite").saveAsTable("my_catalog.bronze.partner_o
rders")
# Only CTAS if you need to transform the shared data through Silver/Gold
layers
```

DI-04: Catalog Federation Over Query Federation

Rule

When Lakehouse Federation supports both query federation (JDBC pushdown) and catalog federation (direct storage access) for a source, prefer catalog federation. It is more performant and cost-effective because queries run entirely on Databricks compute against the source's object storage.

Why It Matters

- **Query federation** pushes SQL to the foreign database via JDBC — limited by JDBC driver capabilities, source compute resources, and network round-trips
- **Catalog federation** reads directly from the foreign catalog's storage layer — queries run fully on Databricks compute with all Spark/Photon optimizations
- Catalog federation avoids loading the source system with queries

Supported Sources

Source	Query Federation (JDBC)	Catalog Federation (Direct Storage)
Snowflake	Yes	Yes
Salesforce Data 360	Yes	Yes
Legacy Hive Metastore (internal)	No	Yes
External Hive Metastore	No	Yes
OneLake	No	Yes
PostgreSQL	Yes	No
MySQL	Yes	No
SQL Server	Yes	No
Oracle	Yes	No
Redshift	Yes	No
BigQuery	Yes	No

Source	Query Federation (JDBC)	Catalog Federation (Direct Storage)
Teradata	Yes	No
Synapse	Yes	No

Correct

```
-- Catalog federation: Snowflake (direct storage access - preferred)
CREATE CONNECTION snowflake_catalog TYPE SNOWFLAKE
OPTIONS (
    host 'myorg.snowflakecomputing.com',
    sfWarehouse 'COMPUTE_WH',
    user secret('federation-scope', 'sf-user'),
    password secret('federation-scope', 'sf-password')
);

CREATE FOREIGN CATALOG snowflake_analytics
USING CONNECTION snowflake_catalog;
-- Queries run on Databricks compute against Snowflake storage
```

Anti-Pattern

```
-- LESS OPTIMAL: Query federation for Snowflake when catalog federation is
available
-- This pushes queries to Snowflake compute via JDBC, adding latency and
cost
-- Use catalog federation instead for better performance
```

DI-05: Transition Federation to Ingestion

Rule

Federation is a bridge, not a destination for production analytics. When access patterns stabilize — query frequency increases, users depend on the data for recurring reports, or joins with local Delta tables emerge — transition from federation to ingestion.

Why It Matters

- Federation costs compound — every query incurs JDBC overhead and source system load
- Ingested data can be optimized with clustering, quality expectations, and caching
- Production SLAs require predictable performance — federation latency varies with source system load

Transition Triggers

Signal	Action
Query frequency exceeds 10/day on a federated table	Ingest — the table is being used regularly
Federated table appears in a scheduled job or dashboard	Ingest — this is production use
Users join federated table with local Delta tables	Ingest — cross-system joins are slow
Source team complains about query load from federation	Ingest — you're impacting their system
Federated table size exceeds 1M rows per query	Ingest — volume exceeds federation sweet spot

Migration Pattern

```
# Step 1: Identify federated tables ready for ingestion
federated_tables = spark.sql("""
    SELECT table_catalog, table_schema, table_name
    FROM system.information_schema.tables
    WHERE table_type = 'FOREIGN'
""").collect()

# Step 2: Create ingestion pipeline for each (example: DLT)
# Move the federated table into Bronze, then build Silver/Gold layers
```

```
# Step 3: Replace federation with Lakeflow Connect
resources:
  pipelines:
    crm_ingest:
      name: "[${bundle.target}] CRM Ingest"
      serverless: true
      catalog: ${var.catalog}
      target: bronze
      ingestion_definition:
        connection_name: legacy_crm  # Reuse the same connection
        objects:
          - table:
              source_schema: public
              source_table: customers
```

DI-06: Document Data Access Pattern per Source

Rule

Maintain a Data Access Registry that records the chosen pattern (ingest, federate, or share) for every external data source, along with the justification, SLA requirements, refresh cadence, and estimated cost.

Review the registry quarterly.

Why It Matters

- Prevents ad-hoc decisions that accumulate technical debt
- Enables cost optimization by identifying sources that should transition from federation to ingestion
- Provides audit evidence for governance and compliance reviews
- Facilitates capacity planning by understanding data flow dependencies

Data Access Registry Template

Source System	Pattern	Justification	SLA	Refresh	Owner	Cost (Monthly)	Last Reviewed
Salesforce	Ingest (Lakeflow Connect)	Production dashboards, >100 queries/day	99.9%	15 min	DE Team	$X	2026-02
Legacy PostgreSQL	Federate (JDBC)	Migration in progress, ad-hoc only	Best effort	Live	Platform	$Y	2026-02
Partner BU Sales	Share (D2D)	Cross-org, partner controls data	Per SLA	Live	Data Ops	$0 (no replication)	2026-02
Snowflake DW	Federate (Catalog) → Ingest	Transitioning — stable patterns identified	TBD	Live → Daily	DE Team	$Z	2026-02

DI-07: Never Federate High-Volume Joins or Aggregations

Rule

Never execute joins between federated foreign tables and local Delta tables in production workloads. Never run large aggregations (GROUP BY with millions of rows) on federated sources. These operations cannot be pushed down efficiently and result in full table scans over the network.

Why It Matters

- JDBC pushdown is limited to simple filters and projections — complex operations fall back to pulling entire tables over the network
- Cross-system joins (federated + local Delta) always pull the federated side entirely, then join locally
- Large aggregations on federated sources consume JDBC connection time and source system resources
- Performance degrades non-linearly as data volume grows

Anti-Pattern

```sql
-- WRONG: Joining federated table with local Delta table
-- This pulls ALL rows from the federated table over JDBC, then joins
locally
SELECT d.customer_name, f.total_orders, f.lifetime_value
FROM crm_postgres.public.customers d
JOIN gold.fact_customer_metrics f ON d.customer_id = f.customer_id
WHERE d.region = 'EMEA';

-- WRONG: Large aggregation on federated source
-- GROUP BY with millions of rows over JDBC — extremely slow
SELECT product_category, COUNT(*), SUM(revenue)
FROM snowflake_fed.analytics.fact_sales
GROUP BY 1;
```

Correct

```sql
-- RIGHT: Ingest the external table first, then join locally
-- Step 1: Ingest (via Lakeflow Connect or DLT)
-- Step 2: Join ingested Delta tables — full Photon optimization
SELECT d.customer_name, f.total_orders, f.lifetime_value
FROM gold.dim_customer d
JOIN gold.fact_customer_metrics f ON d.customer_key = f.customer_key
WHERE d.region = 'EMEA';
```

Validation Checklist

Per-Source Assessment

- [ ] DI-06: Source documented in Data Access Registry with pattern, justification, SLA
- [ ] DI-01: Production workloads use ingested Delta tables (not federation)
- [ ] DI-02: Federated sources are ad-hoc/exploration only (no dashboards, no recurring jobs)
- [ ] DI-03: Cross-org data uses Delta Sharing (not manual export/import)
- [ ] DI-04: Catalog federation used where available (Snowflake, Salesforce, Hive, OneLake)
- [ ] DI-07: No cross-system joins between federated and local Delta in production

Quarterly Review

- [ ] DI-05: Federated sources reviewed for transition to ingestion
- [ ] DI-06: Data Access Registry reviewed and updated
- [ ] Cost analysis: federation vs ingestion cost comparison for each source
- [ ] Source system owners consulted on federation query load

---

References

- Lakeflow Connect (Ingestion)

- Lakehouse Federation
- Delta Sharing
- Lakeflow Connect Managed Connectors
- Auto Loader